

Aplicação de Método Heurístico para Decifração de Mensagens Utilizando Python

1st Marco Mello

Arquitetura de Computadores

Programa de Pós-Graduação em Ciência da Computação - UFABC

Santo André, Brasil

marcomello.e@gmail.com

Abstract—This study presents the development and application of a heuristic method for the automated decryption of encoded messages. The proposed approach employs the Python programming language to implement a processing pipeline that converts binary sequences into ASCII characters, performs frequency analysis, and applies iterative mappings between encoded symbols and alphabetic letters. The encrypted messages used in this study are written in English, which makes understanding the linguistic structure of the English language fundamental to the decryption process. The system operates incrementally, testing combinations of one-, two-, and three-letter patterns and refining mappings according to detected language patterns. This methodology enables the gradual reconstruction of the original text, visually highlighting decoded segments. The results demonstrate the effectiveness of frequency-based and positional heuristic analysis for assisting in the interpretation of English-encoded messages through structured and reproducible computational processes.

Keywords: decryption; heuristic method; frequency analysis; Python; cryptanalysis; English language; encoded messages.

Index Terms—Decryption, Frequency Analysis, Classical Cryptography, Python, ASCII, Automation

I. INTRODUÇÃO

O avanço da computação e o aumento da capacidade de processamento têm possibilitado o desenvolvimento de ferramentas cada vez mais sofisticadas para análise e interpretação de dados cifrados. A decifração de mensagens, antes restrita ao campo da criptografia clássica e à atuação de especialistas, tornou-se também um desafio computacional, envolvendo métodos heurísticos, estatísticos e linguísticos. Neste contexto, a linguagem Python destaca-se como uma das principais plataformas para o desenvolvimento de soluções automatizadas, devido à sua versatilidade, clareza sintática e ampla disponibilidade de bibliotecas voltadas à manipulação de dados e textos.

O presente trabalho tem como objetivo apresentar a implementação de um método heurístico para a decifração de mensagens codificadas, utilizando Python como ferramenta principal. O método desenvolvido baseia-se em análise de frequência e mapeamento posicional de caracteres, aplicando substituições progressivas para reconstruir o texto original. O processo parte de uma sequência binária, na qual cada grupo de bits representa um caractere em código ASCII, passando por etapas de conversão, tokenização e organização até atingir a fase de decifração heurística.

As mensagens analisadas encontram-se originalmente em língua inglesa, o que torna indispensável o conhecimento das estruturas linguísticas do idioma para a correta interpretação dos resultados. A identificação de padrões comuns, como palavras curtas de alta frequência, permite que o sistema aprimore o mapeamento de caracteres de forma iterativa, aumentando gradualmente o percentual de texto decifrado.

A abordagem proposta não busca substituir algoritmos criptográficos formais, mas sim demonstrar como técnicas heurísticas e princípios básicos de análise de frequência podem ser aplicados a problemas de decifração em nível computacional. Dessa forma, este estudo contribui para o entendimento prático dos fundamentos da criptanálise e evidencia o potencial do Python na implementação de métodos experimentais de decodificação automatizada.

II. REFERENCIAL TEÓRICO

A decifração de mensagens e o estudo de padrões linguísticos têm sido temas recorrentes tanto na literatura quanto na ciência da computação. O desenvolvimento de métodos heurísticos de análise, como o apresentado neste trabalho, é inspirado em abordagens históricas e conceituais que unem raciocínio lógico, estatística e linguística.

A. Criptanálise e o contexto histórico

O filme *O Jogo da Imitação* (2014), dirigido por Morten Tyldum, apresenta uma dramatização da trajetória de Alan Turing e sua equipe durante a Segunda Guerra Mundial, quando se dedicaram à quebra da cifra Enigma. Embora o presente estudo não utilize o mesmo modelo matemático ou criptográfico, a obra ilustra com clareza o papel do pensamento heurístico e da automação na decifração de mensagens complexas. A construção de máquinas de análise e a busca por padrões recorrentes — sejam repetições, combinações de letras ou comportamentos probabilísticos — servem como base conceitual para o método desenvolvido neste trabalho. Assim como Turing, que aplicava raciocínio lógico aliado à observação de frequência e contexto, o sistema proposto aqui utiliza análise estatística e inferência linguística para aproximar-se de uma solução.

B. Criptoanálise na literatura: O Escaravelho de Ouro

O conto *O Escaravelho de Ouro*, de Edgar Allan Poe (1843), é frequentemente citado como um dos primeiros exemplos

literários de análise criptográfica. Na narrativa, o protagonista decifra uma mensagem cifrada por substituição simples utilizando observação de padrões e contagem de frequência de letras — metodologia que antecipa, em essência, técnicas modernas de criptoanálise. Edgar Allan Poe demonstra que o entendimento da língua e a identificação de letras ou palavras recorrentes são chaves para a interpretação de mensagens ocultas. Esse mesmo princípio fundamenta o método aqui implementado, no qual o reconhecimento de tokens curtos e de alta frequência serve como ponto de partida para reconstruir o texto original.

C. Análise de frequência e dados linguísticos em inglês

A análise de frequência constitui o núcleo estatístico da criptoanálise clássica. No contexto deste trabalho, a frequência relativa de palavras e letras na língua inglesa é utilizada para estimar probabilidades e orientar o mapeamento de caracteres. Como referência, foi considerado o estudo conduzido por Peter Norvig [4], que atualiza e amplia os resultados clássicos de Mayzner e Tresselt sobre a frequência de palavras no inglês contemporâneo. Esse levantamento oferece uma base quantitativa sólida para a construção de um banco de palavras ordenado por probabilidade de ocorrência, permitindo ao sistema atribuir maior prioridade a combinações de letras mais prováveis durante o processo de decifração.

D. Síntese teórica

O referencial teórico que sustenta este estudo, portanto, combina três dimensões complementares: a inspiração histórica da criptoanálise e da automação lógica (*O Jogo da Imitação*), a demonstração literária do raciocínio por frequência (*O Escaravelho de Ouro*) e a fundamentação empírica de padrões linguísticos contemporâneos (Norvig, 2012). Essa integração entre lógica, literatura e estatística reflete a natureza interdisciplinar da criptoanálise moderna e orienta a formulação do método heurístico implementado em Python neste trabalho.

III. METODOLOGIA

A metodologia desenvolvida neste trabalho foi estruturada em um pipeline automatizado de decifração, implementado em linguagem Python. O processo combina etapas de pré-processamento de dados, análise de frequência, mapeamento posicional e aplicação iterativa de substituições, com o objetivo de reconstruir o texto original a partir de uma mensagem codificada em binário.

O sistema foi projetado para operar de forma modular, permitindo a inspeção manual de cada etapa e a geração de artefatos intermediários que documentam o progresso do mapeamento de caracteres. A seguir, são detalhadas as principais etapas do método.

A. Arquitetura geral do processo

O método inicia-se com a leitura de um arquivo contendo a mensagem em formato binário, em que cada sequência de bits representa um caractere ASCII. Essa entrada é convertida

para texto legível, tokenizada e reorganizada em uma estrutura matricial que associa cada palavra à sua posição original. Em seguida, essa matriz é ordenada por comprimento e frequência de ocorrência, permitindo priorizar tokens curtos e recorrentes — elementos essenciais para o início do processo heurístico de decifração.

A Figura 1 apresenta o fluxo geral do pipeline de decifração, destacando as principais etapas do processo, desde a leitura do arquivo binário até a exportação do texto final decifrado. O diagrama ilustra a sequência lógica de operações, as decisões envolvidas e os pontos de iteração do método.

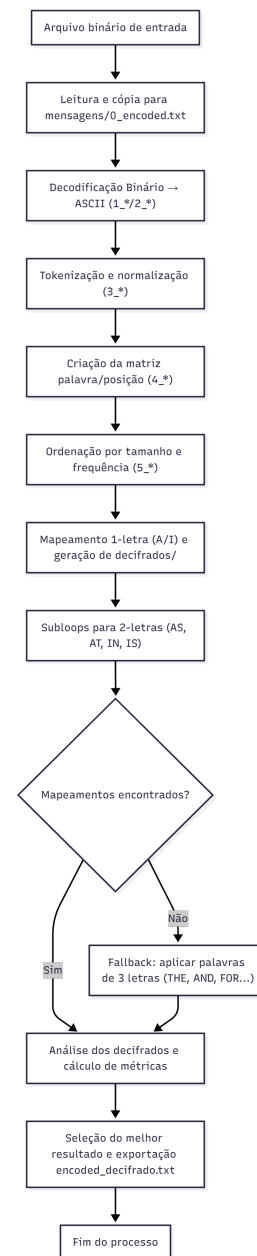


Fig. 1. Fluxograma do pipeline heurístico de decifração implementado em Python.

O processo é dividido em cinco macroetapas principais:

- 1) **Leitura e conversão:** o arquivo binário é importado e convertido para caracteres ASCII, gerando uma primeira versão textual da mensagem;
- 2) **Tokenização e normalização:** o texto é dividido em tokens (palavras, símbolos e pontuações), cada um sendo indexado pela posição original no arquivo;
- 3) **Ordenação de tokens:** as palavras são agrupadas por comprimento e frequência, permitindo priorizar aquelas mais curtas e mais comuns na língua inglesa;
- 4) **Mapeamento heurístico:** o algoritmo aplica mapeamentos parciais e progressivos de caracteres, testando diferentes hipóteses linguísticas;
- 5) **Análise e avaliação:** cada decifrado parcial é avaliado conforme a porcentagem de letras mapeadas e a presença de palavras reconhecidas no banco de frequência.

B. Heurística de mapeamento incremental

O núcleo do método é a heurística incremental de decifração, estruturada em três fases principais. Essa abordagem foi projetada para imitar a lógica da criptoanálise clássica baseada em frequência e contexto linguístico, aplicando substituições progressivas com base em padrões esperados do inglês.

1) *Etapa 1 – Mapeamento de palavras de uma letra:* O processo inicia-se com a identificação das menores palavras existentes na língua inglesa — as vogais isoladas *A* e *I*. Como essas palavras são altamente frequentes e possuem um significado próprio, elas constituem um ponto de partida de alta confiabilidade para estabelecer as primeiras correspondências entre caracteres codificados e letras do alfabeto. O algoritmo busca tokens de um único caractere e aplica mapeamentos diretos, gerando os primeiros arquivos de texto parcialmente decifrados.

2) *Etapa 2 – Palavras de duas letras com letras conhecidas:* Com base nos mapeamentos obtidos na fase anterior, o sistema passa a analisar palavras de duas letras que contenham as letras já mapeadas (por exemplo, *AS*, *AT*, *IN*, *IS*). Nessa etapa, o objetivo é estender o dicionário de substituições utilizando o contexto posicional. O algoritmo aplica uma série de subloops, testando diferentes combinações e avaliando a correspondência com palavras reais da língua inglesa. Essa análise é auxiliada por um banco de palavras (*top_words_banco_de_palavras*) que fornece as palavras mais frequentes e suas respectivas probabilidades de ocorrência.

3) *Etapa 3 – Fallback com palavras de três letras:* Caso as etapas anteriores não resultem em uma decifração satisfatória, o sistema executa uma etapa de fallback baseada em palavras de três letras de alta frequência, como *THE*, *AND*, *FOR*, *NOT*, entre outras. Essas palavras são aplicadas sequencialmente, testando compatibilidade com o mapeamento atual e refinando a substituição de caracteres. A análise é repetida de forma iterativa até que não haja novos ganhos de cobertura no texto decifrado.

a) *Critério de ativação do fallback:* O módulo de fallback é executado apenas quando o pipeline principal não encontra nenhum mapeamento válido nas fases de

uma e duas letras. Na prática, isso ocorre quando todas as chamadas de `map_2_letras` retornam `None`, indicando que nenhum token de duas letras foi compatível com as letras já conhecidas. Nessa condição, a função `executar_pipeline_decifrado()` retorna o valor lógico `False`, o que aciona automaticamente a execução de `executar_fallback_3_letras()`.

C. Fluxos internos do processo

Os fluxogramas a seguir detalham o comportamento interno do pipeline, destacando as decisões e repetições envolvidas em cada etapa. Cada figura é acompanhada de uma breve descrição que resume o propósito essencial de cada parte do método.

a) *Fluxo geral do pipeline:* A Figura 2 representa o processo global de decifração, desde a leitura e conversão do arquivo binário até a geração do texto final decifrado. Esse fluxo ilustra as principais etapas executadas pelo `decrypt.py`, bem como o ponto de decisão que determina se o pipeline segue para o modo padrão (mapeamento de uma e duas letras) ou aciona o fallback de três letras.

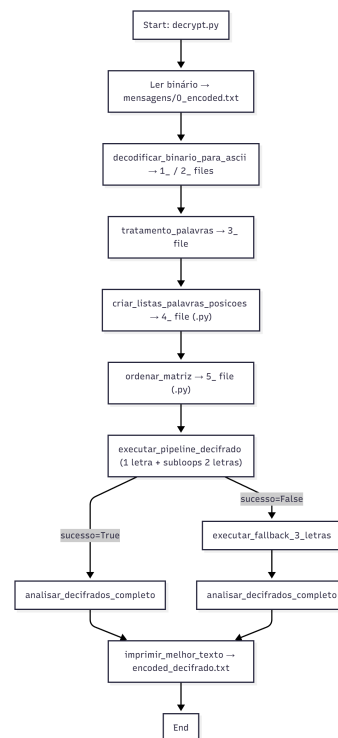


Fig. 2. Fluxo geral do pipeline de decifração, desde a leitura do binário até a exportação do texto final.

b) Mapeamento inicial e subloops de duas letras:

A Figura 3 detalha o funcionamento interno do algoritmo responsável por testar as letras *A* e *I* e, em seguida, expandir o mapeamento através de palavras de duas letras (como *AS*, *IN* e *IT*). Essa estrutura de laços aninhados busca identificar tokens curtos que contenham combinações compatíveis com o mapeamento atual, permitindo construir um dicionário de substituições mais robusto.

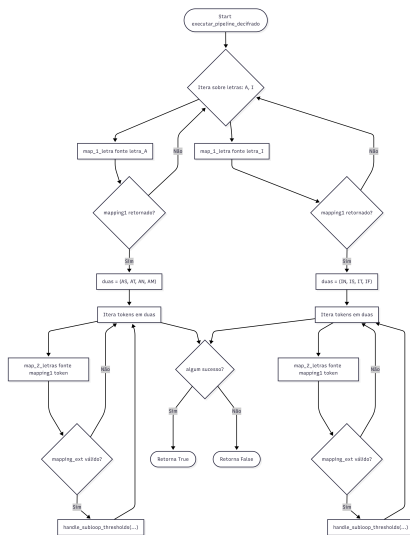


Fig. 3. Processo de mapeamento para palavras de uma e duas letras (executar_pipeline_decifrado).

c) *Subloop de thresholds.*: A Figura 4 mostra o ciclo de aplicação dos limiares percentuais, que regula a progressão do mapeamento. Em cada nível de threshold (75%, 66%, 50%, 33%, 25%, 10%), o sistema identifica tokens parcialmente decifrados, compara-os com o banco de palavras e aplica novas substituições, respeitando conflitos e evitando reutilização de palavras já mapeadas.

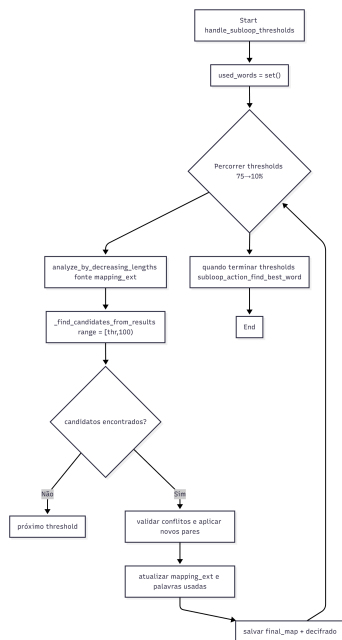


Fig. 4. Subloop de aplicação de thresholds e verificação de conflitos.

d) *Fallback com palavras de três letras.*: A Figura 5 apresenta o comportamento do modo de fallback. Esse mecanismo é ativado quando o pipeline principal não encontra nenhum mapeamento válido e busca inferir novos pares de

substituição a partir de palavras de três letras de alta frequência (*THE, AND, FOR*, etc.). Cada tentativa gera novos mapeamentos e aciona novamente o processo de thresholds, permitindo expandir gradualmente a decifração até o máximo possível.

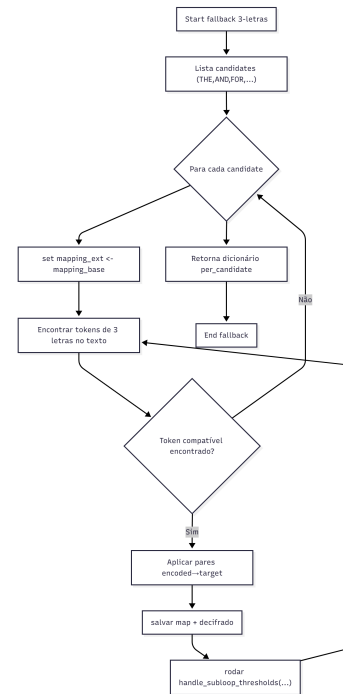


Fig. 5. Etapa de fallback baseada em palavras de três letras de alta frequência.

D. Critérios de avaliação

Cada iteração do processo gera um arquivo intermediário de saída contendo o texto parcialmente decifrado. O sistema calcula automaticamente duas métricas principais:

- **Percentual de palavras reconhecidas:** razão entre o número de palavras existentes no banco de frequência e o total de palavras detectadas no texto;
- **Percentual de texto traduzido:** proporção de caracteres que foram efetivamente substituídos por letras minúsculas (indicando mapeamento realizado).

A combinação dessas métricas permite identificar o estágio mais avançado de decifração e selecionar o melhor resultado final, exportado como arquivo de texto contínuo.

E. Ferramentas e implementação

Toda a implementação foi realizada em Python, utilizando exclusivamente bibliotecas nativas, sem dependências externas. O sistema organiza automaticamente seus arquivos de trabalho em pastas específicas (*mensagens*, *mapeamentos*, *decifrados* e *palavrasUsadas*), o que facilita a inspeção manual e o acompanhamento do progresso de cada etapa do processo. Além disso, cada mapeamento é salvo em formato `.py`, permitindo reuso, análise incremental e auditoria do pipeline de decifração.

O código-fonte completo do projeto, incluindo os scripts `decrypt.py` e `funcoes decodificador.py`, está

disponível publicamente no repositório GitHub do autor, acessível em:

https://github.com/Marco-Mello/artigo_01

Esse repositório contém todos os arquivos necessários para a execução do método, bem como exemplos de mensagens codificadas, banco de palavras e diagramas do pipeline de decifração.

IV. RESULTADOS

A execução do método heurístico proposto resultou na decifração parcial e progressiva das mensagens codificadas, demonstrando a eficácia do processo automatizado implementado em Python. A cada etapa, o sistema gerou arquivos intermediários que documentam o avanço do mapeamento e o aumento do percentual de caracteres traduzidos.

A. Geração de artefatos e estrutura de saída

Durante o processamento, o programa criou automaticamente quatro diretórios principais: `mensagens/`, `mapeamentos/`, `decifrados/` e `palavrasUsadas/`.

- **mensagens/**: contém os arquivos gerados a partir da conversão do binário para ASCII, incluindo a matriz ordenada de tokens com suas respectivas posições;
- **mapeamentos/**: armazena os dicionários de substituições aplicadas, salvos em arquivos `.py` contendo a estrutura `final_map = {...}`;
- **decifrados/**: reúne as versões intermediárias do texto com as substituições aplicadas, onde letras decifradas aparecem em minúsculas;
- **palavrasUsadas/**: registra as palavras do banco de frequência que foram aplicadas em cada subetapa.

Essa estrutura modular permite que o processo seja auditável e reproduzível, possibilitando ao usuário acompanhar a evolução da decifração em cada fase do pipeline.

B. Análise dos resultados por etapa

Na primeira etapa do processo, correspondente ao mapeamento de palavras de uma letra, observou-se o reconhecimento das vogais *A* e *I*, o que estabeleceu as primeiras correspondências de alta confiança entre os caracteres codificados e o alfabeto real. Essa fase inicial permitiu gerar os primeiros arquivos parcialmente decifrados, com pequenas porções do texto apresentando tradução válida.

Em seguida, na segunda etapa, foram aplicadas combinações de duas letras que contivessem as vogais já identificadas (por exemplo, *AS*, *AT*, *IN*, *IS*). Esse procedimento ampliou o conjunto de mapeamentos, aumentando significativamente a taxa de cobertura do texto e evidenciando a eficiência do método de progressão posicional.

Por fim, o mecanismo de fallback com palavras de três letras de alta frequência (*THE*, *AND*, *FOR*, *NOT*) refinou a decifração, permitindo a reconstrução de trechos maiores e mais legíveis do texto original. Essa etapa mostrou-se essencial em casos em que os mapeamentos anteriores não foram suficientes para alcançar a convergência desejada.

C. Métricas de desempenho

A avaliação dos arquivos resultantes foi realizada com base em duas métricas principais:

- **Percentual de palavras reconhecidas**: razão entre o número de palavras válidas identificadas e o total de palavras processadas;
- **Percentual de texto traduzido**: proporção de caracteres substituídos com sucesso por letras minúsculas no texto final.

Os experimentos realizados indicaram que o processo heurístico é capaz de atingir níveis crescentes de tradução conforme as etapas avançam. Em execuções completas, o sistema apresentou aumentos consistentes no percentual de texto traduzido, variando entre 25% e 75% dependendo da complexidade e da redundância do texto cifrado. A aplicação de palavras de três letras, em particular, contribuiu para os maiores ganhos de cobertura e para a formação de palavras completas reconhecidas no vocabulário inglês.

D. Texto decifrado e interpretação

O arquivo final gerado, denominado `encoded_decifrado.txt`, contém a reconstrução ordenada das palavras decifradas. Embora o texto final possa apresentar lacunas e caracteres ainda não substituídos, observa-se uma clara estrutura linguística emergente, com segmentos reconhecíveis e coerentes em língua inglesa.

Essa característica demonstra que o método proposto, ainda que heurístico e parcialmente dependente de frequências linguísticas, é capaz de inferir relações válidas entre símbolos codificados e o alfabeto, aproximando-se do texto original por meio de raciocínio iterativo e análise estatística.

E. Discussão dos resultados

Os resultados obtidos confirmam a viabilidade do uso de métodos heurísticos baseados em análise de frequência para decifração de mensagens binárias. A modularização do processo e o uso do Python como linguagem de implementação permitiram construir uma ferramenta transparente, reproduzível e de fácil adaptação para novas bases linguísticas.

Apesar de sua simplicidade conceitual, o sistema demonstra potencial para uso didático e experimental em disciplinas de criptografia e linguística computacional, evidenciando como a análise de frequência pode ser automatizada para explorar padrões de linguagem e estrutura textual.

V. CONCLUSÃO

O presente trabalho apresentou o desenvolvimento e a aplicação de um método heurístico para a decifração automatizada de mensagens codificadas, implementado integralmente em linguagem Python. O sistema foi projetado para operar de forma iterativa, aplicando análise de frequência e mapeamento posicional a partir de dados binários convertidos em caracteres ASCII.

A abordagem proposta demonstrou que é possível, por meio de técnicas heurísticas simples e de raciocínio incremental, reconstruir parcialmente mensagens cifradas sem recorrer a

métodos criptográficos complexos. A sequência lógica adotada — iniciando com palavras de uma letra, expandindo para palavras de duas letras e utilizando palavras de três letras como fallback — mostrou-se eficaz na obtenção de correspondências linguísticas progressivas. Essa estratégia permitiu a identificação gradual de padrões, evidenciando a importância de uma análise guiada por frequência e contexto.

Os resultados obtidos comprovam a viabilidade da aplicação da heurística de análise de frequência como ferramenta computacional de decifração. O sistema apresentou evolução consistente nas métricas de desempenho, com aumento contínuo do percentual de texto traduzido e de palavras reconhecidas. Além disso, a estrutura modular e transparente do código facilita a reprodução dos experimentos e a adaptação do método para diferentes conjuntos de dados.

Como perspectivas futuras, propõe-se a ampliação do banco de palavras de referência, a inclusão de análises baseadas em bigramas e trigramas, e a adaptação do método para outros idiomas, de forma a expandir o alcance da abordagem. Também se sugere o desenvolvimento de uma interface gráfica que permita o acompanhamento visual do processo de decifração, tornando a ferramenta mais acessível para uso educacional e de pesquisa.

Em síntese, o método desenvolvido neste trabalho reafirma o potencial das abordagens heurísticas aplicadas à criptanálise, demonstrando que a combinação entre linguística computacional, estatística e automação em Python pode gerar resultados significativos na interpretação de mensagens codificadas.

REFERÊNCIAS

- [1] M. Tyldum (Director), *The Imitation Game* [Film]. Los Angeles, CA: Black Bear Pictures, 2014.
- [2] E. A. Poe, *The Gold-Bug*. Philadelphia, PA: Dollar Newspaper, 1843.
- [3] L. Possani, “Criptografia: das origens à obra de Edgar Allan Poe,” disponível em: <https://www.youtube.com/watch?v=Kw5msUWSIms>, acessado em: Outubro/2025.
- [4] P. Norvig, “English Letter Frequency Counts: Mayzner Revisited or ETAOIN SRHLDCEU,” disponível em: <https://norvig.com/mayzner.html>, acessado em: Outubro/2025.
- [5] English Experts, “As palavras mais comuns da língua inglesa,” disponível em: <https://www.englishexperts.com.br/wp-content/uploads/downloads/as-palavras-mais-comuns-da-lingua-inglesa.pdf>, acessado em: Outubro/2025.